

REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
UNIVERSITY OF BUEA
BUEA, SOUTH-WEST REGION



RÉPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
UNIVERSITÉ DE BUEA
BUEA, RÉGION DU SUD-OUEST

Group 23

FACULTY OF ENGINEERING TECHNOLOGY

Report on the

Design and Frontend Implementation of A QoE Application

Course Title: Internet Programming and Mobile App Development

Course code: CEF440

Course Instructor: Dr Nkemeni Valery

Date: June, 2026

Group 23 Members

NAME	MATRICULE
1. THELMA MARLEY NDAYINU	FE23A171
2. EKWEMBWE NKWELLE JUNIOR	FE23A044
3. MBUH ASSURANCE MULUH PILA	FE23A091
4. NYUH DELBERT KIMBI	FE23A129
5. YASIRI TEMWA GASTON	FE23A175

1.0	Introduction	4
1.1	Background	4
1.2	Objectives of This Report.....	4
2.	App Identity.....	5
2.1	Application Name.....	5
2.3	Color Philosophy.....	6
3.	Visual Design	7
3.1	Design Principles and Rationale	7
3.2	Color Tokens and Palette	8
3.3	Typography Specifications	8
3.4	Layout, Spacing, and Component Design Rules.....	9
3.5	Screen-by-Screen Interface Descriptions.....	10
3.6	Screen Design Overview	11
3.6.1	Onboarding Screens.....	11
3.6.2	Subscriber Dashboard Screens	16
3.6.3	Subscriber Feedback Screens	17
3.6.4	Subscriber History Screen.....	18
3.6.5	Subscriber Settings Screen.....	19
3.6.6	Periodic Feedback prompts View	20
4.	Frontend Implementation.....	21

4.1 Native Screen and Composable Structure.....	21
4.2 State Management and Architecture Pattern (MVVM)	21
4.3 Navigation Graph and Routing Model.....	22
4.4 Technical Mapping: Domain and Structural Models to UI Components.....	22

1.0 Introduction

1.1 Background

The frontend implementation phase focuses on transforming the visual designs and system requirements into functional user interface components. The QoE-Monitor frontend was implemented using Kotlin and Jetpack Compose following the MVVM and Clean Architecture pattern defined during the system modelling phase.

The QoE-Monitor project established in Tasks 1 through 4 has produced a complete architectural foundation for a native Android application capable of collecting real-time Quality of Experience data from mobile network subscribers in Cameroon. Task 4 delivered the full system modelling and design phase, producing six formal diagrams that define the system from external context through to physical deployment. Task 5 translates that architectural blueprint into tangible reality: the application now has an identity, a visual design language, and a frontend implementation.

The application is named **netLink**. This name was chosen to reflect the core purpose of the system which is; linking mobile network subscribers in Cameroon to a more transparent, data-driven experience of their own connectivity. The name is short, memorable, and accessible to a broad Cameroonian audience regardless of language background. It communicates connection and network awareness without resorting to technical jargon.

This report documents three distinct but closely related deliverables. App Identity establishes the brand foundation: the name, logo, color philosophy, and the personality that netLink presents to its users. Visual Design translates that brand into a concrete design system: the specific typography, color tokens, component specifications, and screen-by-screen wireframe descriptions that define how every part of the application looks and behaves. Frontend Implementation describes how that design system is realized in code using Jetpack Compose, covering the screen architecture, composable structure, state management approach, and navigation graph.

Together these three deliverables complete the transition from system model to working software and lay the foundation for the database design and full implementation work of Task 6.

1.2 Objectives of This Report

This report covers three primary objectives corresponding to the three deliverables of Task 5.

The first objective is to establish the complete brand identity of netLink, including the application name rationale, logo design, color philosophy, and the tone and personality of the application as experienced by each of its three actor types: Mobile Subscriber, Network Operator, and Administrator.

The second objective is to define the complete visual design system, including all color tokens, typography specifications, component design rules, spacing conventions, and screen-by-screen interface descriptions covering authentication, onboarding, dashboard, feedback, history, and settings screens.

The third objective is to describe the frontend implementation architecture, including the Jetpack Compose screen structure, ViewModel and state management design, navigation graph, and the mapping between the class diagram produced in Task 4 and the Compose UI layer that surfaces those classes to users.

2. App Identity

2.1 Application Name

The application is named **netLink**. The name is intentionally written in lowercase camel case **net** in lowercase, **Link** capitalized to convey approachability and modernity. The lowercase opening signals that this is a tool for everyday people, not a corporate platform. The capitalized Link draws the eye and communicates the core value proposition: this application is a link between the subscriber and the true quality of their network.

The name was evaluated against three criteria specific to the Cameroonian mobile context. First, it must be pronounceable and recognizable across both Anglophone and Francophone regions of Cameroon. netLink satisfies this requirement as both syllables are common in French and English, and the word requires no translation. Second, it must not be confused with an operator brand. netLink is deliberately operator-neutral, reflecting the system's position as an independent subscriber-side tool rather than an operator product. Third, it must communicate the application's purpose without requiring explanation. When a subscriber sees netLink on their device, the name alone conveys that the application is about their network connection.

2.2 Logo Design

The logomark is a stylized signal arc composed of three concentric curved lines of increasing thickness, arranged in the classic WiFi or signal broadcast shape, but with the bottom anchor point replaced by a small filled circle representing the subscriber's device. The arcs are rendered in the primary brand color, Forest Teal (#0D6B5E), against a Cloud White (#FFFFFF) background. The circle anchor uses the accent color, Living Coral (#E8593A), creating a visual focal point that draws attention to the subscriber as the center of the system.

The wordmark renders netLink in Syne ExtraBold 800 at display scale. The net portion is rendered in Deep Slate (#12263A) and the Link portion is rendered in Forest Teal

(#0D6B5E), creating a two-tone typographic treatment that reinforces the brand color association without requiring color in the logomark.

The logo is designed to work at four sizes: full lockup (logomark + wordmark side by side) for onboarding and splash screens, stacked lockup (logomark above wordmark) for dashboard headers, logomark only at 36px for in-app navigation, and logomark only at 20px for notification icons and small UI contexts.



2.3 Color Philosophy

The netLink color system is built around a primary palette rooted in Forest Teal (#0D6B5E), a color chosen deliberately for its associations with trust, clarity, and the natural environment. In the Cameroonian context, teal-green carries warmth without the clinical coldness of pure blue, and it differentiates netLink from the yellow of MTN and the orange of Orange Cameroon, ensuring the application reads as independent and neutral.

The full color token system is defined in Section 3.1. At the identity level, three colors define the netLink brand personality. Forest Teal is the primary identity color: it appears in headers, primary buttons, active states, and the logo. Living Coral (#E8593A) is the action color: it signals calls to action, prompts, and alerts that require the subscriber's attention. Warm Cream (#F5F7F6) is the environment color: it forms the background of every screen, creating a warm, non-sterile reading environment that feels personal rather than institutional.

2.5 Application Personality and Tone

netLink is designed to feel warm, honest, and specifically Cameroonian. It is not a generic global application reskinned for a local market. The copy uses plain language with zero

technical words at the subscriber level. Phrases like "netLink has been keeping watch" and "Join Cameroonians improving mobile networks" are specific, warm, and purposeful.

3. Visual Design

The implementation emphasizes:

- Simplicity and usability
- Low user interaction overhead
- Privacy awareness
- Battery efficiency
- Accessibility
- Real-time responsiveness

The visual design system of netLink translates the application's brand identity into an implementation-ready graphical specification. The interface minimizes user interaction overhead, incorporates strict privacy awareness, and ensures high scannability across Light and Dark system configurations. It avoids technical jargon, communicating cellular behaviors through everyday language tailored specifically for mobile subscribers in Cameroon.

3.1 Design Principles and Rationale

The user experience (UX) and user interface (UI) choices are governed by specific design rationales designed to lower participation barriers and foster user trust:

- **The Anonymous Model over Traditional Authentication:** Traditional "login/setup screens" are replaced entirely with a 5-step onboarding flow. This avoids account creation friction (no passwords, names, or emails) and reinforces data privacy, displaying a short 8-character device ID slice at completion to show functionality without exposing raw UUID strings.
- **Plain-Language Communication:** Technical academic terminology is systematically replaced with descriptive alternatives to maximize inclusivity across non-technical groups.
 - "*Objective readings*" becomes "**Automatic readings**".
 - "*Subjective QoE*" becomes "**Your ratings**".
 - "*Latency*" becomes "**Speed**" or "**Response time**".
 - "*dBm metrics*" maps to easily scannable descriptions ("**Strong**", "**OK**", or "**Weak**").
- **Dual Feedback Paths:** A clear distinction is made between immediate event-triggered situations and reflective user entries. When a network failure happens, a quick-response overlay sheet prompts the user for a single overall rating to avoid interruption. Conversely, a comprehensive 5-category layout is reserved for voluntary entries when the subscriber has free time.
- **Data Integrity via Reading Verification Links:** To ensure collected data remains useful to network operators, ratings are never captured in a vacuum. Every submission records a FeedbackNetworkLink, snapping a background picture of the exact signal metrics (network_metric_id, type, and speed) at that moment.
- **Subscriber APK Refinement:** All development tools, preview buttons, and experimental operator features are fully stripped from the application layer. This

maintains focus on the subscriber and pushes advanced operational charts to a separate web dashboard.

3.2 Color Tokens and Palette

The color palette establishes an independent brand identity separate from major local telecommunication operations (avoiding MTN's yellow or Orange's orange). High contrast balances outdoor legibility under bright sunlight with low-light visual comfort.

- **Primary Brand Tokens (Teal):**
 - TealPrimary (#14B8A6 / #0D6B5E): Base token for button fills, icons, and active accent indicators.
 - TealLight (#5EEAD4): Brand variant used for active accents and gradients within Dark Mode.
 - TealDark (#0D9488): Primary brand color for headers, typography, and text elements in Light Mode.
- **Neutral Tone Tokens (Slate):**
 - Slate900 (#0F172A): Deep canvas backing dark mode layouts.
 - Slate800 (#1E293B): Dark mode container surface for cards and sheet components.
 - Slate700 (#334155): Dark mode surface variant for secondary control elements.
 - Slate600 (#475569): Secondary text coloring for light mode environments.
 - Slate400 (#94A3B8): Secondary text description coloring for dark mode layouts.
 - Slate200 (#E2E8F0): Outlines, dividers, and component borders.
 - Slate50 / Warm Cream (#F8FAFC / #F5F7F6): Non-glare background base canvas.
 - White (#FFFFFF): Light mode surface background for elevated card structures.
- **Semantic Quality Indicators:**
 - QualityGood (#22C55E): Green indicator for strong signals or high satisfaction entries.
 - QualityFair (#F59E0B): Amber indicator for intermediate states or mild signal degradation.
 - QualityPoor (#EF4444): Red indicator for poor cellular reception, low speeds, or complete connection drops.

3.3 Typography Specifications

Typography is bound to the native Android engine default font family (Roboto) to optimize interface loading performance and minimize installation size. Scales utilize scalable pixels (sp) to respect system-wide accessibility adjustments:

Typography Token	Font Size	Font Weight	Contextual UI Mapping
headlineLarge	32sp	Bold	App name on onboarding welcome views
headlineMedium	26sp	Bold	High-visibility hero readouts (e.g., "4G", status directions)
titleLarge	20sp	SemiBold	Main screen titles and structural section headers
titleMedium	16sp	SemiBold	UI Card component title definitions
bodyLarge	16sp	Normal	Informational paragraphs and intro descriptions
bodyMedium	14sp	Normal	Secondary text details and explicit inline help tags
labelLarge	14sp	Medium	Interactive button text, pills, and onboarding step numbers
labelMedium	12sp	Medium	Layout captions, list timestamps, and bottom nav item text labels

3.4 Layout, Spacing, and Component Design Rules

Spatial uniformity across layouts is strictly enforced via fixed Jetpack Compose sizing configurations:

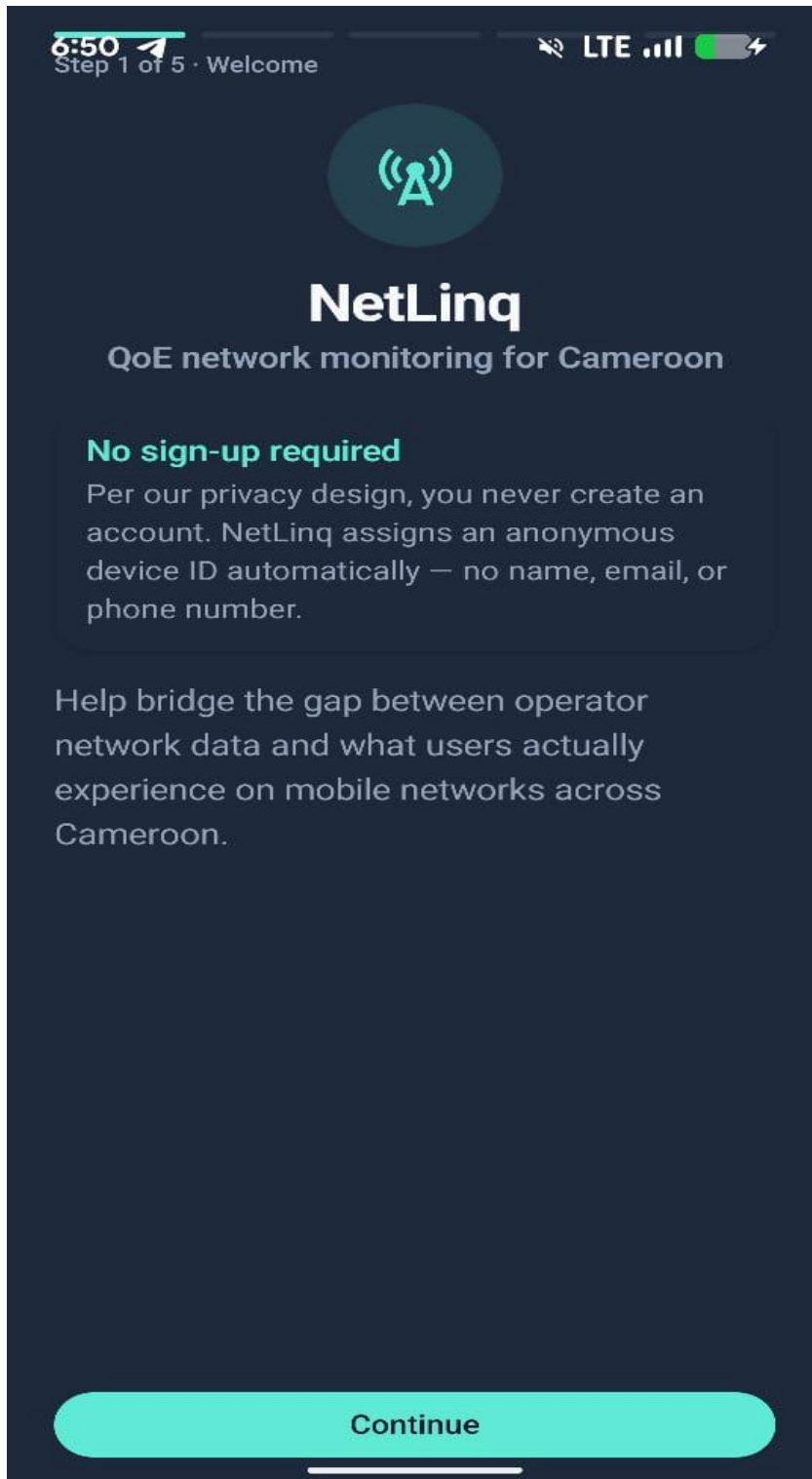
- **Margins and Lateral Edge Padding:** Screen layouts use uniform horizontal side gutters of 20–24 dp.
- **Vertical Section Spacing:** Layout sections and card groups use a standard vertical spacing gap of 16–20 dp to keep content organized.
- **Component Architecture (The NetLinqCard):** Reusable information groupings utilize a custom card structure with a corner radius of 20 dp and an elevation depth of 3 dp.
- **Touch Input Optimization:** Interactivity scales to clear accessibility standards. Interactive 5-star rating bars use custom hit-boxes spanning 36–40 dp to eliminate misclicks or input frustration on smaller screens.
- **Edge-to-Edge Windows:** Application layouts leverage `statusBarsPadding()` rules to draw content behind the system status and navigation areas without clipping readable labels.

3.5 Screen-by-Screen Interface Descriptions

1. **Onboarding (5-Step Sequential Wizard):** Gathers user consent and access permissions without an authentication gate. Steps follow a clear hierarchy: Step 1 (Welcome and Brand Introduction), Step 2 (Plain-Language Feature Overviews), Step 3 (Three Required Consent Checkboxes), Step 4 (Hardware Permissions), and Step 5 (Ready state with anonymous device ID preview).
2. **Network Dashboard:** Acts as the central app view. If no data is present, it uses a clean layout cantered on a "Tap Check now below" directive. When populated, it shows a primary Hero Card styled with a subtle teal gradient, showing connection quality pills, network types, and MetricStatChips (Signal, Speed, Type) beside manual checking controls.
3. **Your History:** Formatted around local offline-first Room data arrays, tracking user entries over time. This screen highlights rolling speed trend charts, cellular category breakdowns, and a historic log mapping user entries directly to the signal snapshots captured during submission.
4. **Feedback Form (Detailed Feedback):** Accessed via the persistent third bottom navigation tab. It features 5 clear evaluation rows with 5-star bars to track Overall experience, Internet responsiveness, Streaming, Call quality, and General satisfaction. The submission button remains disabled until a rating is selected for all 5 categories.
5. **Contextual Feedback Prompts (Quick Rating Sheet):** A responsive bottom sheet that slides up when background loops identify network events, such as signal drops or connection loss. It presents a single 5-star overall rating bar for quick, single-tap submissions.
6. **Application Settings:** Organized by priority to give users clear control. Controls are arranged vertically: monitoring toggles, background collection switches, monitoring triggers, feedback cooldown frequency, theme choice selection (System default / Light / Dark), and a concise data privacy overview.

3.6 Screen Design Overview

3.6.1 Onboarding Screens



How NetLinq works

Two types of data combine to show real Quality of Experience.



Objective monitoring

Signal strength, network type (2G–5G/WiFi), and latency — measured on your device.



Subjective feedback

Quick star ratings on calls, streaming, and overall satisfaction when issues appear.



Offline-first

Data is stored locally first, then synced anonymously when you have internet.



Privacy by design

Uploaded records are anonymized. Operators see aggregated trends, not your identity.

Back

Continue

Your consent

We only start after you agree to each of these — you stay in control.



Network monitoring

Collect signal, latency, and connection type while you use the app.



Feedback notifications

Receive short prompts when network quality changes or issues are detected.



Anonymous data sync

Upload anonymized readings to help improve network insights (WiFi by default).

You can stop monitoring or change sync settings anytime in Settings.

Back

Continue

App permissions

Each permission has a clear purpose. Nothing extra is requested.



Phone state

Read signal strength and cellular network type (2G, 3G, 4G, 5G).



Location

Required by Android to access cell tower signal information. We do not track your GPS position.



Notifications

Show quick QoE feedback prompts when network issues are detected.



Network access

Measure latency and sync anonymized data when online.

Permissions granted

Back

Continue

6:50
Step 5 of 5 · Ready

LTE



You're ready

Setup is complete. No account was created — that's intentional.

Anonymous device ID

3f892046...

This ID is hashed before any upload. It cannot identify you personally.

Next: your dashboard shows network readings.
The Feedback tab is for your subjective ratings.

Back

Open dashboard

3.6.2 Subscriber Dashboard Screens



3.6.3 Subscriber Feedback Screens

Rate your experience

Quick subjective check — takes under 30 seconds.
We may also prompt you when the network changes.

Current network: 4G

Overall experience

How was your network just now?



Internet responsiveness

Pages, messages, and apps loading quickly?



Streaming

Video and music playback quality?



Call quality

Voice and video calls clear?



General satisfaction

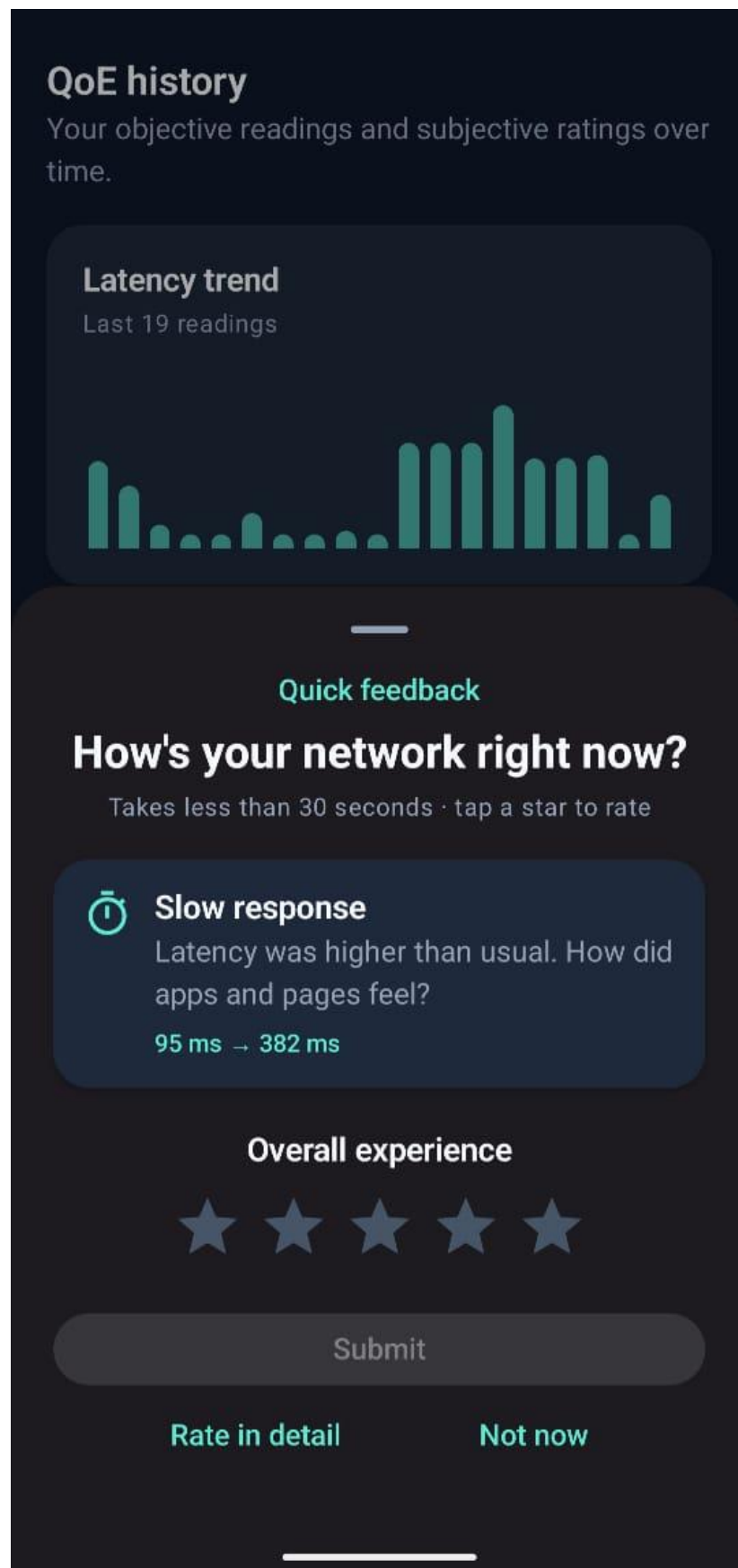
Would you recommend this network right now?



Notes (optional)

Submit feedback

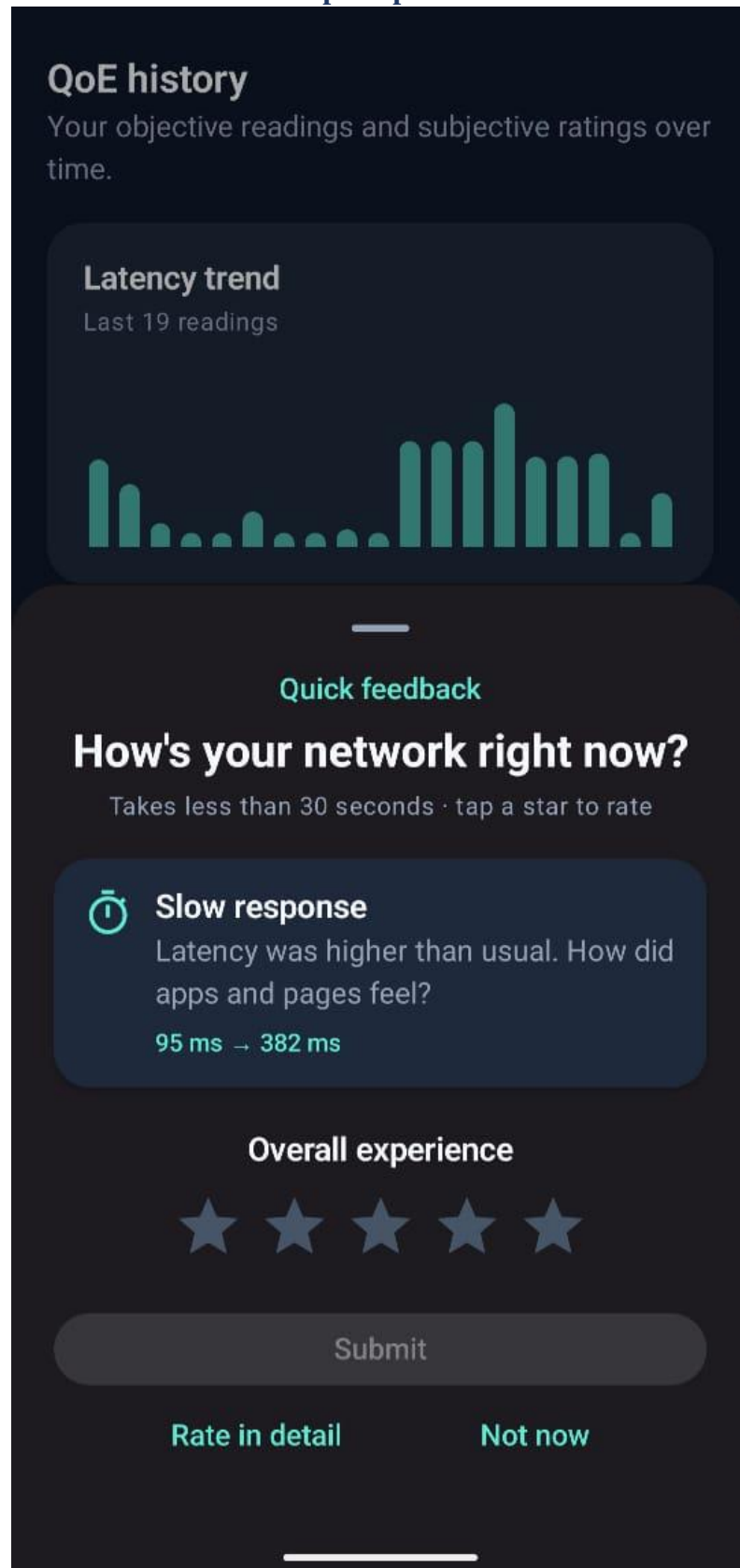
3.6.4 Subscriber History Screen



3.6.5 Subscriber Settings Screen



3.6.6 Periodic Feedback prompts View



4. Frontend Implementation

The frontend architecture developed for netLink translates abstract design guidelines into a native, performance-tuned Android application built entirely with Jetpack Compose.

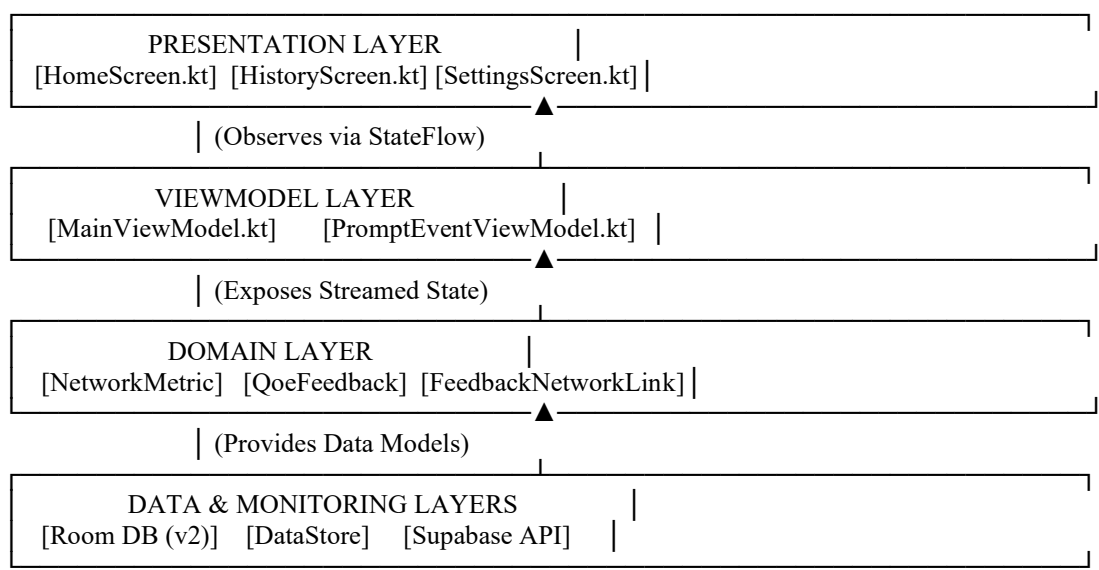
4.1 Native Screen and Composable Structure

The user interface layer is built with declarative composable functions (.kt files), shifting layout definitions out of static XML files and directly into clean, reactive Kotlin code.

- **Application Root Scaffold (MainScaffold.kt):** This manages the global application frame, overriding default Material 3 styling assumptions to provide a consistent look across both themes. It handles window expansions and coordinates the transitions of the central view components.
- **Bespoke Navigation Bar (NetLinkBottomBar):** Replaces the standard Material purple layout with a container color matching the application card surfaces. Active tabs use a clean teal accent highlighted by a soft teal capsule badge. The navigation bar hosts 4 core tabs: Dashboard (using a brand-tied cell tower icon), History, Feedback, and Settings.

4.2 State Management and Architecture Pattern (MVVM)

The source architecture relies on the **Model-View-ViewModel (MVVM)** pattern, establishing clean separation boundaries among UI layouts, application states, business constraints, and data repositories.



- **Stateless Component Design:** Individual composable layouts are built as stateless components whenever possible. They receive immutable state values as inputs and

pass user interactions up to view controllers using standard Kotlin lambda execution parameters.

- **Reactive Flow Channels:** ViewModels (such as `MainViewModel.kt` and `PromptEventViewModel.kt`) pull updates from local data engines and background network monitors. They convert these entries into safe, lifecycle-aware `StateFlow` streams observed within the UI using native `collectAsState()` collection calls to optimize battery usage and memory conservation.

4.3 Navigation Graph and Routing Model

Screen destinations, arguments, and transitions across different views are defined programmatically inside `NetLinqNavGraph.kt`.

- **Onboarding Navigation Gating:** At application launch, the entry routing path queries the local preferences storage (`DataStore`) via `MainViewModel`. If the onboarding initialization flag evaluates to false, the system blocks tab interaction and routes the subscriber into the 5-step onboarding view flow. Once onboarding is completed, these setup screens are cleared from the backstack to prevent users from accidentally returning to the welcome flow via the back button.
- **Intent-Driven Notification Routing:** Background monitoring triggers utilize explicit Android system intent extras to handle deep-linking interactions. Tapping a local notification launches `MainActivity`, which extracts specific `network_metric_id` parameters and passes them directly to the navigation controller. This displays the modal feedback panel pre-populated with relevant signal snapshots.

4.4 Technical Mapping: Domain and Structural Models to UI Components

The logical data models defined during initial object modeling tasks directly govern how information is formatted and displayed on the screen:

- **NetworkMetric Component Mapping:** Telemetry gathered in the background via hardware hooks (such as `TelephonyManager`) maps directly onto domain data entities. These structures feed the custom `MetricStatChips` component blocks on the frontend, translating technical figures into readable descriptions like Strong, OK, or Weak.
- **QoeFeedback Data Execution:** Captures subscriber entries across the 5 evaluation categories. View states inside `FeedbackScreen.kt` monitor this data in real time, disabling the primary submission button until all 5 structural domains are rated.
- **FeedbackNetworkLink & Room v2 Architecture:** To ensure subjective user ratings are connected with objective network data, the local database was refactored into **Version 2**. The frontend reads this combined entity structure via `HistoryScreen.kt`, displaying user satisfaction data directly alongside snapshots of signal strength and speed captured at the moment of submission. This unified structure ensures highly contextual data maps accurately for future aggregation and analysis.